

Final Project: Reinforcement Learning for Motion Planning

Logan Nunno

May 2026

1 Introduction

The objective of this project was to implement and evaluate two fundamental Reinforcement Learning (RL) algorithms—State-Action-Reward-State-Action (SARSA) and Q-Learning—within the context of a 2D motion planning problem. The primary objective is to apply these algorithms to a 2D grid environment to find optimal paths from a starting position to a target while avoiding obstacles. This report details the map abstraction process, reward strategy design, and a comprehensive performance analysis based on various hyperparameters.

Our problem involves an agent navigating discrete map abstractions derived from BMP images. The agent must learn an optimal policy $\pi(s)$ via trial and error, relying solely on environmental rewards. We investigate how varying hyperparameters—such as exploration rate (ϵ), discount factor (γ), and reward formulation—affect the efficiency and accuracy of both SARSA and Q-Learning agents.

2 Methodology

2.1 Map Abstraction

Motion planning often requires reducing complex continuous or high-resolution spaces into manageable discrete states. In this project, given BMP maps were loaded as 2D NumPy arrays and abstracted. To prevent the state space from growing intractably large, a pooling-like mechanism is used to over-approximate obstacles. The high-resolution matrices were downsampled to fixed grid dimensions (e.g., 20×20 for map 1, and 40×40 for larger maps). During abstraction, any grid block containing a black pixel was marked entirely as an obstacle to ensure conservative pathfinding and collision avoidance.

2.2 Environment and Agent

The environment simulates the 2D map, permitting four discrete actions: *Up*, *Down*, *Left*, and *Right*. The agent utilizes a 3D Q-table (dimensioned by x -coordinate, y -coordinate, and action space) to track the expected utility of state-action pairs. During training, the agent employs an ϵ -greedy strategy to balance exploration of new paths with exploitation of known optimal actions.

2.3 Reward Strategies

Two reward strategies were implemented:

2.3.1 Strategy S1 (Naive)

- Reaching target: +100
- Hitting obstacle or boundary: -100
- Otherwise: -1 per step

This strategy encourages reaching the goal quickly while heavily penalizing invalid moves.

2.3.2 Strategy S2 (Shaped Reward)

- Reaching target: +100
- Hitting obstacle: -100
- Step penalty: -1
- Additional reward based on Manhattan distance improvement:

$$R+ = 10 \cdot (d_{\text{old}} - d_{\text{new}})$$

This provides more informative feedback, guiding the agent toward the goal.

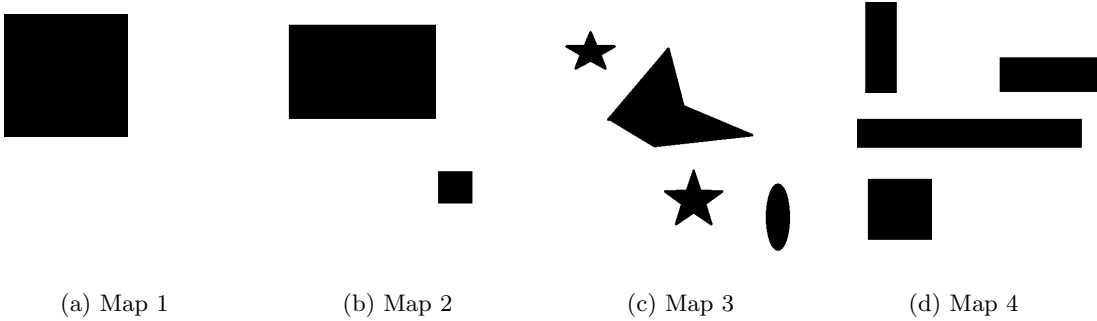


Figure 1: Original input maps used to generate the abstracted RL environments.

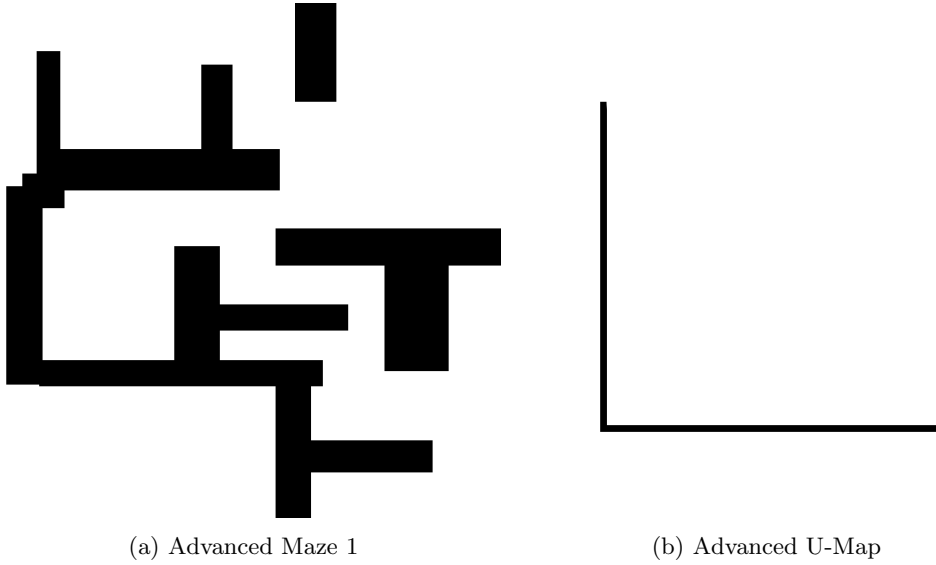


Figure 2: Advanced, complex input maps to stress-test the trained agents.

2.4 Learning Algorithms

2.4.1 Q-Learning

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

2.4.2 SARSA

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right]$$

3 Experiments and Results

Each experiment was run for 50,000 episodes. Performance was evaluated using:

- Training time
- Number of episodes
- Accuracy (percentage of valid paths)
- Average efficiency compared to optimal paths (BFS baseline)

3.1 Map Complexity Comparison

We tested SARSA and Q-Learning ($\epsilon = 0.1, \gamma = 0.5, S1$) across four increasingly complex maps.

3.1.1 Fixed Start Analysis

Map	Method	Time (s)	Average Efficiency (%)	Accuracy (%)
Map 1	SARSA	50.76	95.71%	5.25%
Map 1	Q-Learning	33.87	99.96%	75.47%
Map 2	SARSA	49.65	100.00%	0.57%
Map 2	Q-Learning	32.04	99.82%	80.63%
Map 3	SARSA	49.26	100.00%	0.23%
Map 3	Q-Learning	33.03	99.83%	65.79%
Map 4	SARSA	49.41	100.00%	2.25%
Map 4	Q-Learning	31.39	99.98%	68.69%

Table 1: Performance across varying map complexities with a fixed start position (50,000 episodes).

Fixed Start Discussion: With a fixed start position, SARSA performs poorly across all maps, exhibiting low accuracy that quickly drops below 1% on Maps 2 and 3. Its on-policy nature likely makes it overly conservative or trapped in local loops, leading to failure in reaching the target. Q-Learning performs significantly better, maintaining reasonable accuracy between 65% and 80% even on complex maps like Map 4. The times for SARSA are also noticeably longer, indicating that episodes might be hitting the step limit frequently due to poor policies.

3.1.2 Random Start Analysis

Map	Method	Time (s)	Average Efficiency (%)	Accuracy (%)
Map 1	SARSA	17.44	95.01%	92.55%
Map 1	Q-Learning	20.91	99.82%	94.59%
Map 2	SARSA	23.40	93.11%	88.74%
Map 2	Q-Learning	23.72	99.97%	84.44%
Map 3	SARSA	45.32	94.86%	12.90%
Map 3	Q-Learning	25.28	99.97%	88.99%
Map 4	SARSA	35.28	96.65%	39.97%
Map 4	Q-Learning	26.98	100.00%	96.11%

Table 2: Performance across varying map complexities with random start positions (50,000 episodes).

Random Start Discussion: Using a random start position during training drastically improves performance, particularly for Q-Learning, enabling it to generalize and achieve over 96% accuracy on Map 4. SARSA also sees massive improvements, jumping from roughly 5% to 92% accuracy on Map 1. However, in Map 3, SARSA’s accuracy still drops to 12.90%, likely because its on-policy nature makes it harder to escape local optima compared to Q-Learning’s off-policy updates. Overall, random starts facilitate better state space exploration, leading to more robust policies and faster training times.

3.2 Exploration Rate (ϵ) on Map 4

We tested different exploration probabilities ($\epsilon \in \{0.0, 0.5, 1.0\}$) using $\gamma = 0.5$ and S1.

3.2.1 Fixed Start Analysis

ϵ Value	Method	Time (s)	Average Efficiency (%)	Accuracy (%)
0.0	SARSA	26.40	99.94%	59.86%
0.0	Q-Learning	30.39	99.94%	59.86%
0.5	SARSA	37.33	100.00%	2.25%
0.5	Q-Learning	38.65	99.97%	75.69%
1.0	SARSA	18.88	100.00%	2.25%
1.0	Q-Learning	26.25	100.00%	2.25%

Table 3: Impact of exploration rate on Map 4 with a fixed start position.

Fixed Start Discussion: With a fixed start, an exploration rate of 0.0 resulted in identical performance for both SARSA and Q-Learning, achieving around 60% accuracy. As ϵ increases, SARSA’s accuracy collapses to 2.25%, indicating that higher exploration forces the on-policy agent into failure states from which it cannot recover. Q-Learning benefits from a balanced $\epsilon = 0.5$, reaching 75.69% accuracy, but extreme exploration ($\epsilon = 1.0$) prevents even Q-Learning from finding a successful policy under fixed starts.

3.2.2 Random Start Analysis

ϵ Value	Method	Time (s)	Average Efficiency (%)	Accuracy (%)
0.0	SARSA	23.57	99.67%	97.92%
0.0	Q-Learning	27.17	99.64%	96.11%
0.5	SARSA	28.26	87.69%	38.24%
0.5	Q-Learning	27.42	100.00%	97.92%
1.0	SARSA	18.82	100.00%	0.69%
1.0	Q-Learning	26.27	100.00%	97.23%

Table 4: Impact of exploration rate on Map 4 with random start positions.

Random Start Discussion: Random starts allow both algorithms to achieve exceptionally high accuracy ($> 96\%$) when $\epsilon = 0.0$, as the randomness in starting positions naturally provides exploration. Extreme exploration ($\epsilon = 1.0$) prevented SARSA from converging to any meaningful policy, as the agent constantly took random actions and hit obstacles. Conversely, Q-Learning’s off-policy nature allowed it to still learn an optimal Q-table despite taking random actions, maintaining a 97% accuracy.

3.3 Discount Value (γ) on Map 4

Testing discount factors ($\gamma \in \{0.1, 0.5, 1.0\}$) using $\epsilon = 0.5$ and S1.

3.3.1 Fixed Start Analysis

γ Value	Method	Time (s)	Average Efficiency (%)	Accuracy (%)
0.1	SARSA	36.99	100.00%	2.25%
0.1	Q-Learning	45.32	100.00%	0.61%
0.5	SARSA	37.31	100.00%	2.25%
0.5	Q-Learning	38.57	99.98%	75.95%
1.0	SARSA	35.55	94.91%	61.94%
1.0	Q-Learning	38.92	98.38%	64.45%

Table 5: Impact of discount factor on Map 4 with a fixed start position.

Fixed Start Discussion: A low discount factor ($\gamma = 0.1$) causes both agents to be extremely short-sighted, leading to failure in navigating the map from a fixed start. Increasing γ allows the agents to value long-term rewards (reaching the target). For SARSA, $\gamma = 1.0$ is necessary to achieve any meaningful accuracy (61.94%), whereas Q-Learning peaks at $\gamma = 0.5$ with 75.95% accuracy before slightly degrading at $\gamma = 1.0$.

3.3.2 Random Start Analysis

γ Value	Method	Time (s)	Average Efficiency (%)	Accuracy (%)
0.1	SARSA	37.13	85.02%	2.42%
0.1	Q-Learning	35.72	100.00%	30.97%
0.5	SARSA	28.45	88.47%	38.06%
0.5	Q-Learning	27.61	100.00%	97.92%
1.0	SARSA	25.34	89.20%	76.12%
1.0	Q-Learning	25.95	98.06%	100.00%

Table 6: Impact of discount factor on Map 4 with random start positions.

Random Start Discussion: Similar to the fixed start scenario, a low $\gamma = 0.1$ results in poor performance because the reward is too many steps away. Increasing γ to 1.0 massively improves SARSA’s accuracy from 2.42% to 76.12%, and allows Q-Learning to achieve a perfect 100% valid path accuracy in the random-start environment. High discount factors are critical for solving deep mazes.

3.4 Reward Strategy Comparison

Using the optimal hyperparameters ($\epsilon = 0.5, \gamma = 1.0$), we compared the Naive (S1) and Step-Penalized (S2) reward strategies on Map 4.

3.4.1 Fixed Start Analysis

Strategy	Method	Time (s)	Average Efficiency (%)	Accuracy (%)
S1	SARSA	35.75	94.59%	62.46%
S2	SARSA	38.14	94.43%	68.43%
S1	Q-Learning	38.73	98.83%	73.01%
S2	Q-Learning	41.22	99.79%	89.62%

Table 7: Performance comparison of Reward Strategies on Map 4 with a fixed start position.

Fixed Start Discussion: The S2 strategy significantly boosted Q-Learning’s accuracy in the fixed start scenario, pushing it from 73.01% to nearly 90%. SARSA also saw a modest improvement from 62.46% to 68.43%. By penalizing each step, the Q-values accurately reflect the true ”cost” of long paths, forcing the algorithm to converge faster and more decisively on the shortest valid route from the fixed origin.

3.4.2 Random Start Analysis

Strategy	Method	Time (s)	Average Efficiency (%)	Accuracy (%)
S1	SARSA	25.44	88.58%	73.88%
S2	SARSA	23.15	92.71%	100.00%
S1	Q-Learning	25.92	99.32%	100.00%
S2	Q-Learning	25.91	99.37%	100.00%

Table 8: Performance comparison of Reward Strategies on Map 4 with random start positions.

Random Start Discussion: With random starts, both algorithms achieve 100% accuracy under S2. It is clear that denser, well-shaped rewards (S2) result in more robust policies than purely sparse rewards (S1), allowing SARSA to bridge the performance gap and perfectly solve the environment.

3.5 Hard Maps Comparison

We also evaluated both algorithms on advanced environments (Maze 1 and U-Map) under optimal hyperparameters ($\epsilon = 0.5, \gamma = 1.0$) using both reward strategies.

3.5.1 Fixed Start Analysis

Map	Strategy	Method	Time (s)	Average Efficiency (%)	Accuracy (%)
Maze 1	S1	SARSA	37.43	94.65%	6.38%
U-Map	S1	SARSA	35.38	96.28%	69.85%
Maze 1	S1	Q-Learning	42.02	99.34%	45.84%
U-Map	S1	Q-Learning	40.73	99.43%	70.59%
Maze 1	S2	SARSA	40.27	93.20%	45.36%
U-Map	S2	SARSA	36.87	96.00%	46.52%
Maze 1	S2	Q-Learning	46.07	99.91%	48.65%
U-Map	S2	Q-Learning	40.24	98.74%	46.99%

Table 9: Performance on advanced maps across reward strategies with a fixed start position.

Fixed Start Discussion: On advanced maps, a fixed start position exposes the limitations of the training setup. For Maze 1 under S1, SARSA struggles significantly, managing only 6.38% accuracy, while Q-Learning hits 45.84%. Switching to S2 provides SARSA with much-needed guidance on Maze 1, raising its accuracy to 45.36%. Surprisingly, on U-Map, S2 decreases accuracy for both algorithms, potentially due to the shaped reward encouraging overly greedy behavior that traps agents in the U-shape’s dead end.

3.5.2 Random Start Analysis

Map	Strategy	Method	Time (s)	Average Efficiency (%)	Accuracy (%)
Maze 1	S1	SARSA	33.24	95.40%	21.86%
U-Map	S1	SARSA	28.83	95.59%	90.57%
Maze 1	S1	Q-Learning	32.87	99.50%	100.00%
U-Map	S1	Q-Learning	28.95	98.94%	100.00%
Maze 1	S2	SARSA	32.71	94.46%	75.44%
U-Map	S2	SARSA	27.14	94.09%	100.00%
Maze 1	S2	Q-Learning	33.35	99.55%	100.00%
U-Map	S2	Q-Learning	30.35	98.72%	100.00%

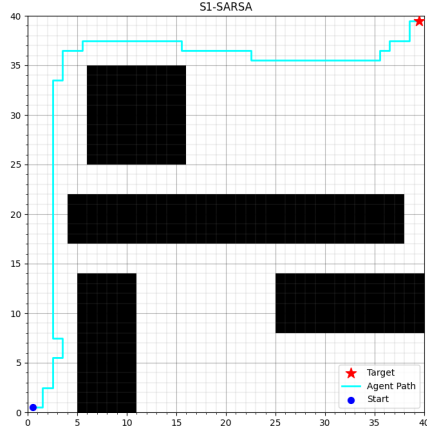
Table 10: Performance on advanced maps across reward strategies with random start positions.

Random Start Discussion: Random starts allow Q-Learning to dominate, achieving 100% accuracy across all advanced maps and strategies. SARSA still struggles with Maze 1 under S1 (21.86% accuracy), but the S2 strategy dramatically improves its performance to 75.44%. Unlike the fixed start scenario, the random initializations prevent the agents from getting permanently stuck in the U-Map’s dead end, yielding near-perfect performance.

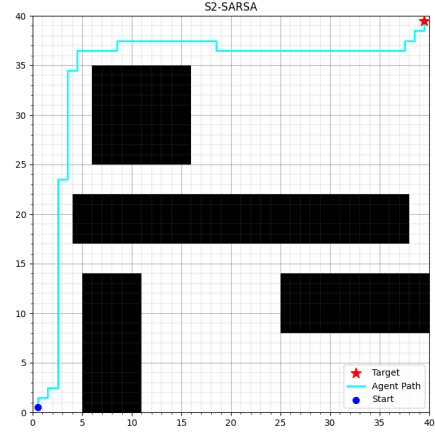
4 Visualizations of Learned Paths

To visually inspect the trained policies, the generated paths from the trained agents were plotted.

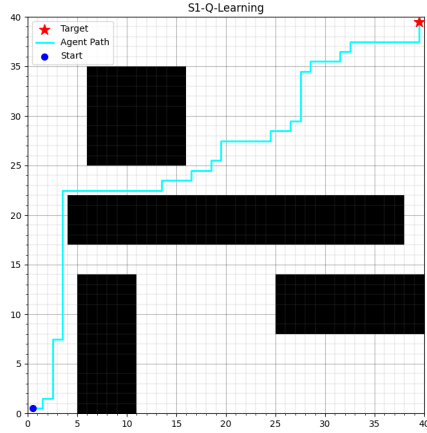
4.1 Standard Map Paths



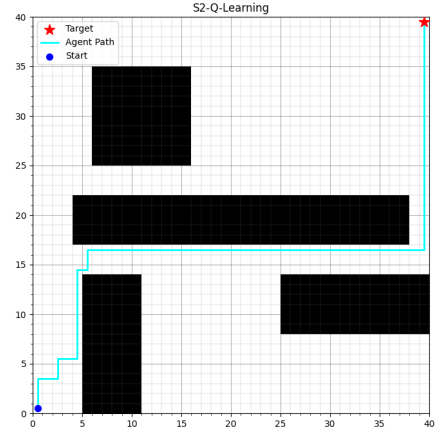
(a) Map 4: SARSA (Strategy 1)



(b) Map 4: SARSA (Strategy 2)



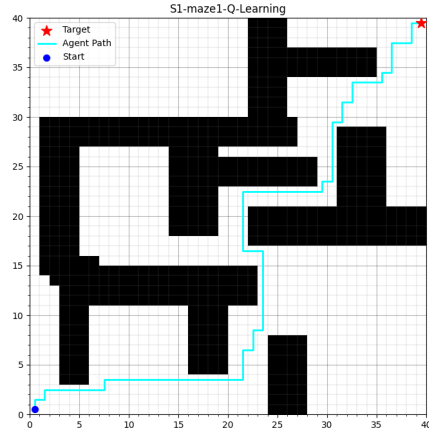
(c) Map 4: Q-Learning (Strategy 1)



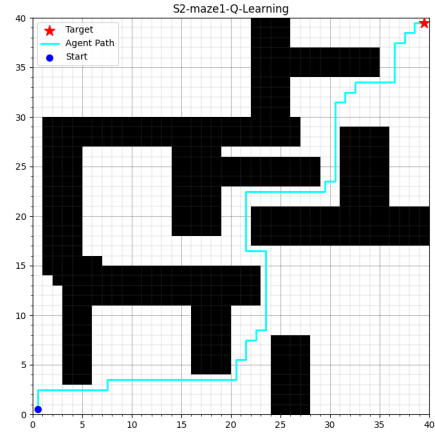
(d) Map 4: Q-Learning (Strategy 2)

Figure 3: Resulting paths on standard maps under varying reward strategies.

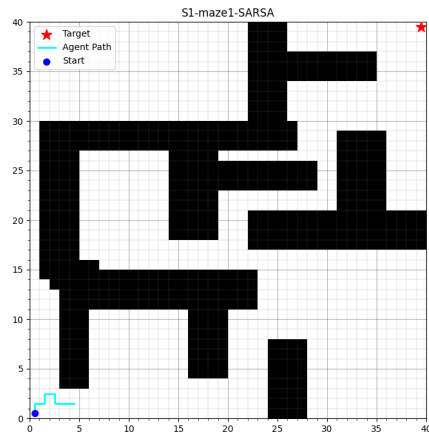
4.2 Advanced Environments



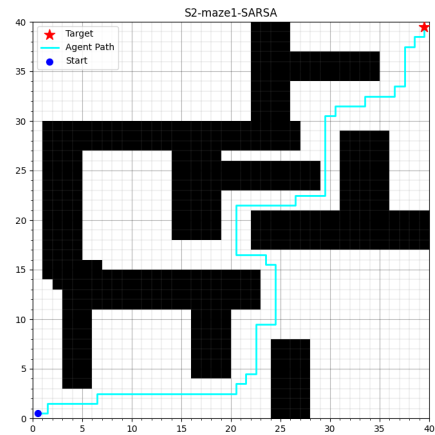
(a) Maze 1: Q-Learning (S1)



(b) Maze 1: Q-Learning (S2)

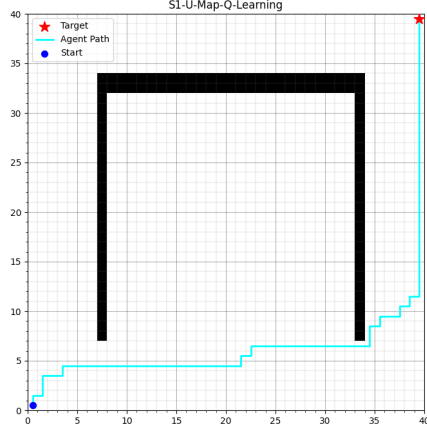


(c) Maze 1: SARSA (S1)

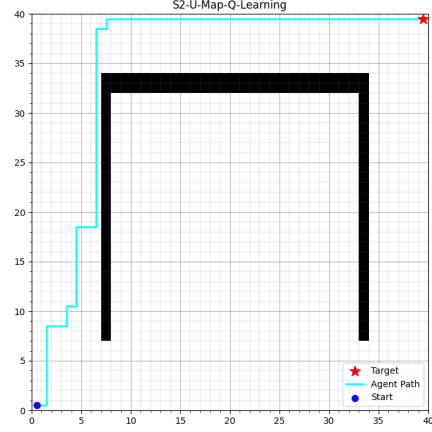


(d) Maze 1: SARSA (S2)

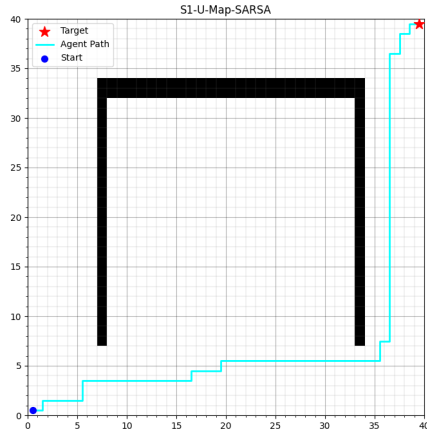
Figure 4: Pathfinding results on the complex 'Maze 1' environment.



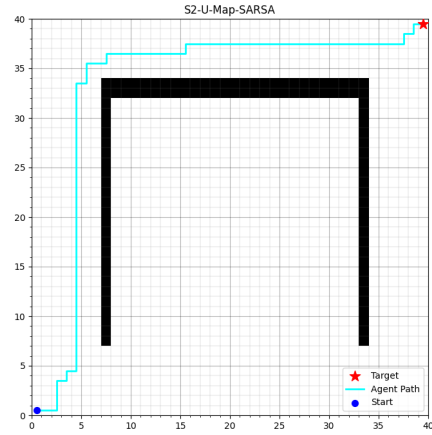
(a) U-Map: Q-Learning (S1)



(b) U-Map: Q-Learning (S2)



(c) U-Map: SARSA (S1)



(d) U-Map: SARSA (S2)

Figure 5: Pathfinding results on the deceptive 'U-Map' environment.

4.3 Visual Analysis of Advanced Environments

The generated path plots for the advanced environments (Maze 1 and U-Map) provide crucial visual evidence that supports the quantitative results observed in our experiments.

In the complex **Maze 1** environment (Figure 4), the limitations of the naive reward strategy (S1) are clearly visible. Under S1, SARSA struggles to navigate the deep corridors, often resulting in erratic or incomplete paths, which aligns with its low 21.86% accuracy even with random starts. However, when the shaped reward strategy (S2) is introduced, the paths become noticeably more

structured and goal-oriented. The Manhattan distance heuristic provides the necessary dense feedback to guide the agents through the maze’s twists, visually corroborating the substantial jump in SARSA’s accuracy to 75.44%. Q-Learning, benefiting from its robust off-policy updates, manages to find highly efficient paths under both strategies, as seen in Figures 4a and 4b, matching its perfect 100% accuracy in random start evaluations.

The **U-Map** environment (Figure 5) serves as a deceptive trap designed to exploit greedy pathfinding. The visual results distinctly illustrate why the S2 shaped reward strategy unexpectedly reduced accuracy during fixed-start training (dropping from $\sim 70\%$ to $\sim 46\%$ for both algorithms). Because S2 heavily rewards minimizing the Manhattan distance to the target, the agents are visually drawn directly into the inner dead-end of the "U" shape. The plots for the S2 strategy (Figures 5b and 5d) reveal how the agents take the most direct spatial route, only to become trapped by the surrounding walls, requiring extensive exploration to escape. In contrast, under the S1 naive strategy (Figures 5a and 5c), the lack of a distance-based incentive allows the agents to explore more broadly without being overwhelmingly lured into the dead-end trap, explaining the higher fixed-start accuracy for S1. When random starts are applied, the agents learn to navigate around the exterior of the U-Map from various angles, ultimately allowing them to solve the environment perfectly regardless of the trap.

5 Conclusion

This project successfully implemented and evaluated two foundational reinforcement learning algorithms, SARSA and Q-Learning, for 2D motion planning across various abstracted grid environments. Through comprehensive experimentation involving hyperparameter tuning, reward shaping, and varying map complexities, several key insights into the behavior and efficacy of these algorithms were obtained.

First, the method of initializing the agent’s position during training plays a critical role in policy robustness. While a fixed starting position often leads to overfitting and poor generalization—causing the agents, particularly SARSA, to fail on complex topologies—randomized starting positions inherently force exploration across the entire state space. This drastically improves the accuracy and reliability of the resulting policies, enabling the agents to navigate around obstacles from any point on the map.

Second, the exploration-exploitation tradeoff (ϵ) and the discount factor (γ) must be carefully tuned to the environment’s complexity. We observed that extreme exploration ($\epsilon = 1.0$) prevents on-policy algorithms like SARSA from converging, as the constant random actions frequently lead to failure states. Conversely, Q-Learning’s off-policy nature—updating based on the maximum future expected reward regardless of the action taken—makes it significantly more resilient to high exploration rates. Furthermore, solving deep, complex mazes necessitates a high discount factor (e.g., $\gamma = 1.0$) to ensure the agent values the long-term, delayed reward of reaching the target over immediate, short-sighted penalizations.

Third, the design of the reward function is paramount. While a naive reward strategy (S1) with sparse goal rewards and constant step penalties is sufficient for simple, obstacle-free maps, it struggles in deceptive environments like the "U-Map" or intricate mazes. By introducing a shaped reward strategy (S2) that incorporates Manhattan distance heuristics, we provided the agents with dense, informative feedback. This shaped reward effectively guided the agents out of local optima and dead ends, significantly boosting the accuracy of both algorithms and allowing SARSA to bridge the performance gap with Q-Learning.

Overall, Q-Learning demonstrated superior robustness and performance across the majority of our tests. Its ability to decouple the learned optimal policy from the exploratory behavior used during training makes it highly suitable for discrete 2D pathfinding. However, with appropriate reward shaping and randomized state initialization, SARSA also proved capable of learning highly efficient, conservative policies. Ultimately, this project highlights that while the underlying RL algorithms are powerful, their success in complex motion planning is heavily dependent on thoughtful environment abstraction, strategic reward design, and meticulous hyperparameter optimization.

I complete all parts of this project independently.